Fundamental Concepts in ROS 2

Shagun Kanani
Formula Manipal Driverless Taskphase
Manipal Institute of Technology
Manipal, Karnataka, India

Abstract*— ROS 2 (Robot Operating System 2) is an open-source software framework for developing distributed robotic applications. It provides communication mechanisms, development tools, libraries, and interfaces that allow individual software components to operate as independent nodes and exchange information. This report presents the fundamental concepts of ROS 2, including the computation graph, nodes, topics, services, actions, parameters, custom interfaces, workspaces, Colcon, and Quality of Service (QoS). The report also describes the implementation of two ROS 2 applications.

Keywords*— ROS 2, Humble, Computation Graph, Nodes, Topics, Services, Actions, Parameters, Custom Interfaces, Colcon, QoS, Robotics

I. Introduction

ROS 2 is an open-source software development framework designed for robotics applications. Rather than requiring an entire robotic application to be implemented as one program, ROS 2 encourages a modular architecture in which different functions are implemented as independent nodes. These nodes communicate using standardised ROS 2 communication mechanisms.

In this task, ROS 2 Humble was selected as the target distribution. The work involved studying the ROS 2 command-line tools, Turtlesim and RQT, nodes, topics, services, parameters, actions, logging, launch systems, workspaces, packages, Colcon, publisher-subscriber communication, custom interfaces, and QoS settings.

II. Fundamental Concepts of ROS2

A. Computational Graphs

The ROS graph is a network of ROS 2 elements processing data together at the same time. It encompasses all executables in nodes and the connections between them. It describes how different nodes exchange information through topics, services, and actions within a ROS 2 system. The graph provides a high-level representation of the data flow and interaction between different components of a robotic application. Command: rqt_graph

B. Nodes

A node is an executable unit within a ROS 2 application that performs a specific logical function. A robotic system is generally divided into multiple nodes so that individual functions can be developed and maintained independently.

Examples include:

- Camera node
- LiDAR node
- Localisation node
- Motor controller node
- Navigation node
- Sensor processing node

A node can contain publishers, subscribers, services, actions, parameters, timers, and other ROS 2 entities required for its operation. Nodes communicate with other nodes through standardised ROS 2 communication mechanisms rather than relying on direct implementation-specific connections.

C. Topics

Topics provide a publisher-subscriber communication mechanism in ROS 2. A publisher sends messages to a named topic, while one or more subscribers can receive and process messages published on that topic. Topics are primarily intended for asynchronous communication and are well suited to continuously generated data such as sensor measurements, camera information, robot states, and velocity data. Multiple publishers and subscribers can participate in topic-based communication, allowing information to be distributed across several nodes.

D. Publishers and Subscribers

A publisher is a ROS 2 entity responsible for generating and transmitting messages through a topic. A subscriber is an entity that receives messages from a topic and processes them using an appropriate callback mechanism. Publisher and subscriber communication is asynchronous, allowing the publishing and receiving nodes to operate independently. This separation enables flexible communication in which individual nodes can be added, removed, or modified without requiring substantial changes to other nodes.

E. Services

Services provide a request-response communication mechanism between ROS 2 nodes. A service consists of a client that sends a request and a server that processes the request and returns a response. Services are therefore appropriate for operations such as resetting a component, requesting a calculation, or triggering a specific state change. They are not generally intended for continuous high-frequency data streams because their communication model is based on individual requests and responses.

F. Actions

Actions are designed for long-running operations that may require feedback during execution before producing a final result. This communication model is useful when an operation cannot be completed immediately, and its progress needs to be monitored. Actions therefore provide functionality beyond simple request-response communication by supporting goals, intermediate feedback, and final results.

G. Parameters

Parameters are configuration values associated with ROS 2 nodes. They allow characteristics of a node to be modified without changing its source code and recompiling the application. Parameters may represent numerical values, strings, Boolean states, operating modes, thresholds, or other configuration information required by a node. They are primarily intended for configuring node behaviour rather than transmitting continuous runtime data between nodes.

H. ROS 2 Client Libraries

ROS 2 provides client libraries that allow developers to create applications and communicate with the ROS 2 middleware using different programming languages. The two principal client libraries are `rclpy` for Python and `rclcpp` for C++. These libraries provide interfaces for creating nodes and implementing publishers, subscribers, services, actions, parameters, timers, and other ROS 2 functionality. The availability of multiple client libraries allows developers to select a programming language according to the requirements of the application.

III. Differences Between Topic and Services

Aspect	Topic	Service
Communication model	Publisher–subscriber	Request–response
Communication nature	Asynchronous and continuous	Request-based and discrete
Main entities	Publisher and Subscriber	Client and Server
Response	No direct response required	Response returned for every request
Data usage	Suitable for continuous data such as sensor readings	Suitable for specific operations such as resetting or configuring a system
Multiple receivers	Multiple subscribers can receive the same data	Request is handled by the server and a response is returned

IV. Turtlesim

Turtlesim is a lightweight simulation environment provided for learning and demonstrating fundamental ROS 2 concepts. It provides a simple graphical environment in which nodes, topics, services, and other ROS 2 communication mechanisms can be observed without requiring physical robotic hardware.

The turtlesim node can be launched using:

```bash
ros2 run turtlesim turtlesim_node
```

The keyboard teleoperation node can be launched using:

```bash
ros2 run turtlesim turtle_teleop_key
```

V. RQT

RQT is a graphical framework for ROS 2 tools and plugins. It provides graphical interfaces for inspecting and debugging ROS 2 systems.

RQT can be launched using:

```bash
rqt
```

The graph visualization tool can be accessed through:

```bash
rqt_graph
```

`rqt_graph` displays the relationships between nodes and the communication interfaces connecting them.

VI. ROS 2 Workspace

A ROS 2 workspace is a directory structure used to organise and develop ROS 2 packages. The source space contains the packages and source code developed by the user, while the build space contains intermediate files generated during the build process. This separation allows source code, generated build artefacts, installed files, and logs to be organised systematically. Workspaces also support the development of custom applications on top of an existing ROS 2 installation.

VII. ROS 2 Packages

A ROS 2 package is the basic organizational unit used to distribute and manage ROS 2 software. A package can contain nodes, source files, custom interfaces, launch files, configuration files, dependencies, and other resources required by an application. Organizing software into packages allows related components to be grouped together while maintaining clear dependency relationships. Packages therefore provide a standardized structure for developing, building, testing, and distributing ROS 2 applications.

VIII. Colcon

Colcon is the build system commonly used for building ROS 2 workspaces and their packages. It identifies packages within a workspace, determines their dependencies, and performs the required build operations in the appropriate order. The build process generates the files required to execute and use the packages within the workspace. Colcon also supports building selected packages and provides options that are useful during iterative software development.

IX. Custom Message Interfaces

ROS 2 provides predefined message types for commonly required data structures, but application-specific systems may require custom data formats. Custom message interfaces allow developers to define the fields and data types required by a particular application. A custom message is defined using a `.msg` interface specification containing one or more fields with their associated data types. Once the interface is built, it can be used by different ROS 2 nodes for communication through topics.

X. Custom Service Interfaces

Custom service interfaces allow developers to define application-specific request and response structures for service communication. It is defined using '.srv' file. A service interface contains a request section and a response section, separated by a delimiter in the service definition. The request specifies the information supplied by the client, while the response specifies the information returned by the server. They allow request-response communication to be designed specifically for the requirements of the robotic system.

XI. Quality of Service

Quality of Service, commonly abbreviated as QoS, defines the communication policies used by ROS 2 entities when exchanging data. QoS allows communication behavior to be adapted according to the requirements of different applications and data streams. Important QoS policies include reliability, durability, history, and depth.

A. Reliability

The reliability policy determines the degree to which message delivery is guaranteed between communicating ROS 2 entities. A reliable communication policy attempts to ensure that messages reach their intended destination, while a best-effort policy prioritizes efficient communication and permits message loss. Reliable communication is useful when losing a message could significantly affect the application, whereas best-effort communication can be appropriate for high-frequency data where occasional loss is acceptable.

B. Durability

Durability determines whether previously published messages are retained for subscribers that join the communication system later. With volatile durability, messages are intended for currently active subscribers and previously published samples are not retained for future subscribers. Transient-local durability allows a publisher to retain relevant samples so that compatible late-joining subscribers can receive them. The appropriate durability policy depends on whether historical information is required by subscribers.

C. History

The history policy determines how message samples are maintained by the communication system. The keep-last policy retains a specified number of the most recent messages, while the keep-all policy attempts to retain messages subject to the available resources.

D. Depth

The depth value determines the number of messages retained when the keep-last policy is used. These policies are important when communication occurs at high frequency and the subscriber may not process messages as quickly as they are generated. Appropriate history and depth settings help balance memory usage, latency, and data availability.

XII. RQT Graphs and Terminal Screenshots

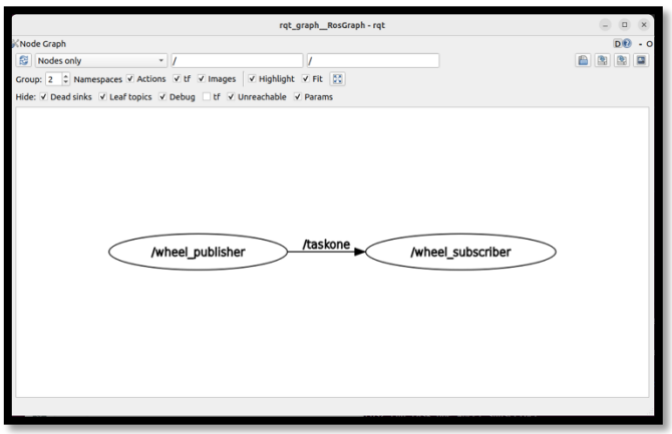
Q1: Wheel Longitudinal Speed Calculation

To calculate the longitudinal speed of a wheel using angular velocity and radius transmitted between two ROS 2 nodes through a custom message.

A. Nodes

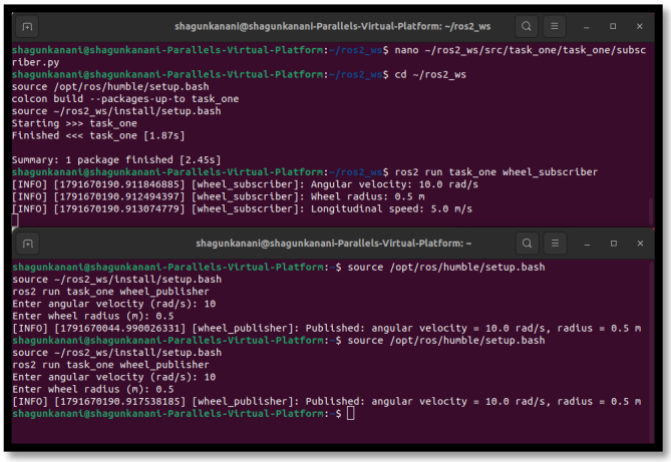
- wheel_publisher: Accepts angular velocity and wheel radius from the terminal and publishes them using the custom TaskOne message on /taskone.
- wheel_subscriber: Receives the message and calculates the longitudinal speed using $v=\omega r$, displaying the result in the terminal.
- The custom message TaskOne.msg contains two float32 fields: ang_vel and radius.

B. RQT Graph



C. Terminal Output

Angular Velocity: 10.0 rad/s
Wheel Radius: 0.5 m
Longitudinal Speed: 5.0 m/s



The implementation successfully demonstrates custom-message communication between two ROS 2 nodes and the longitudinal speed calculation using the received parameters.

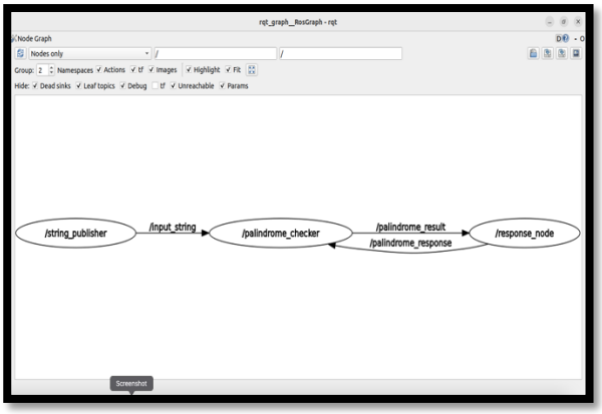
Q2: Palindrome Detection

To develop a three-node ROS 2 system that determines whether a user-provided string is a palindrome and returns a corresponding text response.

A. Nodes

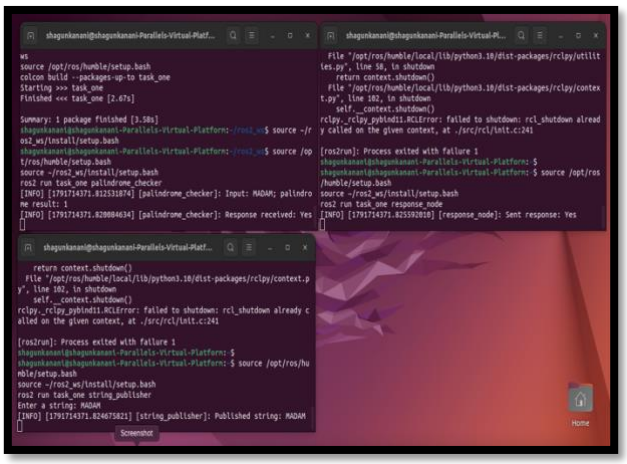
- string_publisher: Publishes user-provided strings on /input_string using custom RT2String message, which contains a single string field named data.
- palindrome_checker: Receives the string, converts it to lowercase, removes spaces, and checks whether it matches its reverse. It publishes 1 for a palindrome or 0 otherwise on /palindrome_result.
- response_node: Receives integer result and publishes "Yes" for 1 or "No" for 0 on /palindrome _response.

B. RQT Graph

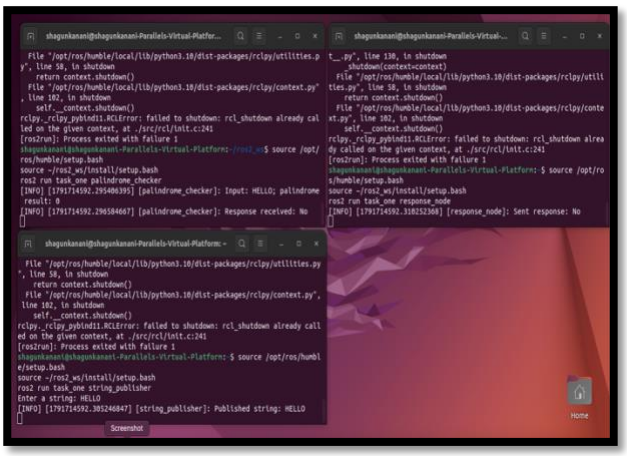


C. Terminal Output

Enter a string: MADAM
Palindrome Result: 1



Enter a string: HELLO
Palindrome Result: 0



The implementation demonstrates multi-node communication, custom interfaces, string processing, conditional logic, and response handling using ROS 2 topics.

References

[1] Open Robotics, “Why ROS 2?,” ROS 2 Documentation.
[2] Open Robotics, “Understanding ROS 2 Topics,” ROS 2 Humble Documentation.
[3] Open Robotics, “Understanding ROS 2 Services,” ROS 2 Humble Documentation.
[4] Open Robotics, “Creating a Workspace,” ROS 2 Humble Documentation.
[5] Open Robotics, “Using colcon to build packages,” ROS 2 Humble Documentation.
[6] Open Robotics, “About Quality of Service Settings,” ROS 2 Documentation.
[7] Open Robotics, “ROS 2 Humble Documentation,” Open Robotics.